OPERATING SYSTEMS (UE25CS242A)

Unit 2 – IPC, Threads, Synchronization and Deadlocks

Question Bank

Lecture 1 – IPC: Shared Memory & Message Passing

1. Differentiate between shared memory and message passing with respect to communication mechanism and kernel involvement.

2. Why is synchronization necessary when multiple processes communicate using shared memory?

3. A multimedia application transfers large video frames between processes. Which IPC mechanism would you choose and why?

4. Differentiate between blocking and non-blocking message passing with suitable examples.

5. Explain the architecture of shared memory communication with a neat diagram.

6. Compare shared memory and message passing based on speed, synchronization, kernel involvement, and security.

7. Illustrate the sequence of communication between two processes using the send() and receive() primitives.

8. Explain how race conditions occur in shared memory communication and discuss suitable synchronization mechanisms to prevent them.

9. Explain the Producer-Consumer problem using the bounded buffer approach. What happens when the buffer is full and when it is empty?

10. Differentiate between direct and indirect communication in message passing. How does a mailbox-based system resolve the issue of multiple receivers?

11. Compare zero-capacity, bounded-capacity, and unbounded-capacity buffering in message passing. In which case must the sender always wait?

12. A weather forecasting system consists of multiple computational processes that frequently exchange large matrices. Analyze whether shared memory or message passing is more appropriate.

13. Design an IPC architecture for an online banking system where multiple independent processes exchange transaction information. Justify the choice of IPC mechanism considering performance, security, and reliability.

14. Compare shared memory and message passing in terms of implementation complexity, scalability, synchronization overhead, fault isolation, security, and practical applications.

15. Design a producer-consumer communication framework using shared memory. Explain how semaphores or mutexes can be used to eliminate race conditions.

16. What are the reasons for process cooperation? List the advantages of cooperating processes.

17. Explain what a rendezvous is in the context of message passing synchronization.

18. Explain how a message-passing system handles the Producer-Consumer problem compared to the shared memory approach. Which approach is simpler to implement?

Lecture 2 – System Calls for IPC: Pipes, FIFOs, UNIX Domain Sockets

1. Differentiate between the functions of shmget() and shmat() in shared memory communication.

2. Why are ordinary pipes suitable only for communication between related processes?

3. Compare ordinary pipes and FIFOs based on process relationship, communication mechanism, and persistence.

4. State any four advantages of UNIX domain sockets over Internet sockets for local communication.

5. Explain the sequence of Linux system calls required to establish communication using System V shared memory (shmget(), shmat(), shmdt(), shmctl()).

6. Write a C program using pipe(), fork(), read(), and write() to transfer a string from a parent process to its child process.

7. Develop a Linux program using mkfifo() to enable communication between two unrelated processes.

8. Compare Shared Memory, Ordinary Pipes, Named Pipes (FIFOs), and UNIX Domain Sockets with respect to communication model, kernel involvement, performance, and synchronization requirements.

9. Trace the execution of a UNIX Domain Socket server from socket creation to client communication, explaining each system call.

10. A programmer forgets to close the unused ends of a pipe after calling fork(). What problems can arise?

11. A FIFO-based communication program hangs indefinitely during execution. State two possible reasons for this behavior.

12. A developer reports that data written into a FIFO is not being received by another process. Analyze possible causes related to synchronization, file permissions, and blocking behavior.

13. Design a client-server application using UNIX Domain Sockets for communication between multiple local processes. Explain the sequence of system calls.

14. A high-performance AI inference server consists of multiple worker processes on the same Linux machine. Evaluate the suitability of Shared Memory, Pipes, FIFOs, and UNIX Domain Sockets for exchanging large volumes of data.

15. Write the sequence of Linux system calls required to establish one-way communication between a parent and child process using an ordinary pipe.

16. Differentiate between the roles of listen() and accept() in UNIX Domain Socket programming.

17. Which system call removes a named FIFO file from the file system and why is this necessary?

Lecture 4 – Threads, Multicore Programming & Multithreading Models

1. Differentiate between a process and a thread with respect to memory organization and execution.

2. List any four resources shared among threads belonging to the same process.

3. Differentiate between User-Level Threads (ULTs) and Kernel-Level Threads (KLTs).

4. A web browser opens separate tabs for different websites. Explain why multithreading is more suitable than creating a separate process for each tab.

5. Explain the architecture of a multithreaded process with a neat diagram showing shared and private resources.

6. Compare single-threaded and multithreaded applications with respect to performance, responsiveness, resource utilization, and scalability.

7. Discuss the challenges associated with multithreaded programming, including race conditions, synchronization overhead, deadlocks, and debugging complexity.

8. Differentiate between the Process Control Block (PCB) and the Thread Control Block (TCB).

9. Compare the Many-to-One, One-to-One, and Many-to-Many multithreading models with advantages and limitations of each.

10. Write the syntax of pthread_create() and explain the purpose of each argument.

11. Why is pthread_join() preferred over using an infinite waiting loop when synchronizing threads?

12. Describe the life cycle of a POSIX thread from creation to termination using pthread_create(), pthread_exit(), and pthread_join().

13. Differentiate between Symmetric Multiprocessing (SMP) and Asymmetric Multiprocessing (AMP).

14. Explain the concept of CPU affinity. Discuss hard affinity and soft affinity with suitable examples.

15. Describe the role of load balancing in multicore scheduling. Explain push migration and pull migration techniques.

16. Explain the concept of Processor Affinity and how it conflicts with Load Balancing in an SMP system.

17. A multithreaded application experiences poor performance because its threads frequently migrate between CPU cores. Which multicore scheduling concept can improve performance?

19. Compare the Many-to-Many and Two-Level multithreading models. What advantage does the Two-Level model provide over Many-to-Many?

18. A multimedia application performs video decoding, audio playback, subtitle rendering, and user interaction simultaneously. Design a multithreaded architecture for this application.

Lecture 5 – Multicore Kernel Concepts

1. Explain why multiprocessor scheduling is more complex than single-processor scheduling.
2. Differentiate between asymmetric multiprocessing and symmetric multiprocessing (SMP).

3. Explain the need for synchronization in a multicore kernel.

4. What is kernel parallelism? Explain its significance in SMP systems.

5. A processor has 4 cores, with 2 hardware threads per core. How many logical CPUs are visible to the operating system? Explain.

6. Explain the difference between push migration and pull migration.

7. Explain why load balancing is necessary in an SMP system.

8. Explain the concept of processor affinity and its relationship with cache performance.

9. Differentiate between soft affinity and hard affinity

Lecture 7 – Mutual Exclusion and Synchronization

1. Define the Critical Section Problem. Why does it occur in concurrent systems?

2. State the three requirements that a correct solution to the Critical Section Problem must satisfy.

3. Differentiate between race condition and critical section.

4. Two processes increment the same shared variable without synchronization. Identify the problem and explain the possible consequence.

5. Explain the Critical Section Problem with a neat diagram showing the Entry Section, Critical Section, Exit Section, and Remainder Section.

6. Describe Peterson's Algorithm for achieving mutual exclusion between two processes. Explain the role of the flag[] and turn variables.

7. Explain how race conditions occur in a multithreaded banking application where multiple threads update the same account balance.

8. Explain the three requirements for solving the Critical Section Problem — Mutual Exclusion, Progress, and Bounded Waiting. Analyze whether Peterson's Algorithm satisfies all three.

Lecture 8 – Principles of concurrency synchronization hardware

1. Define concurrency and explain how it differs from parallelism.

2. Why are atomic operations important in concurrent systems?

3. Differentiate between Test-and-Set (TSL) and Compare-and-Swap (CAS) instructions.

4. Describe the working of the Test-and-Set instruction and explain how it ensures mutual exclusion.

5. Explain busy waiting (spin waiting). Discuss its advantages, disadvantages, and situations where it is preferred.

6. Compare software-based synchronization techniques (Peterson's Algorithm) and hardware-supported mechanisms (Test-and-Set, Compare-and-Swap) in terms of scalability, fairness, and performance.

7. A railway reservation system allows multiple users to book the same seat simultaneously. Analyze how race conditions may occur and design a synchronization solution.

8. A shared counter is updated concurrently by multiple threads without synchronization. Identify the problem and explain why hardware support is necessary.

9. Develop the pseudo-code for implementing a spinlock using the Test-and-Set instruction and discuss the drawbacks of busy waiting.

Lecture 9 – Mutex Locks, Semaphores

1. Differentiate between a mutex lock and a binary semaphore.

2. Explain the purpose of the wait() and signal() operations in semaphore synchronization.

3. When is a spinlock preferred over a mutex lock? Justify your answer.

4. A multithreaded application experiences reduced performance because many threads are waiting for the same lock. Identify the problem and suggest a solution.

5. Explain the working of a mutex lock with a neat diagram. Discuss how it ensures mutual exclusion.

6. Describe the working of counting semaphores and binary semaphores. Compare their applications.

7. Explain spinlocks and discuss their advantages and disadvantages in multicore operating systems.

8. Define lock contention. Explain its causes and discuss techniques to minimize it in multicore systems.

9. A ticket booking system allows multiple users to reserve seats simultaneously. Design a synchronization solution using mutex locks.

10. Compare Mutex Locks, Binary Semaphores, Counting Semaphores, and Spinlocks with respect to ownership, blocking behavior, scalability, and performance.

Lecture 10 – Classical Synchronization Problems

1. State the objective of the Producer-Consumer (Bounded Buffer) Problem.

2. Differentiate between the Readers-Writers Problem and the Producer-Consumer Problem.

3. Why are Reader-Writer Locks more efficient than ordinary mutex locks in read-intensive applications?

4. Explain the Producer-Consumer Problem with a neat diagram. Describe the roles of the producer, consumer, and shared buffer.

5. Develop a semaphore-based solution for the Producer-Consumer Problem using the semaphores mutex, empty, and full.

6. Explain the Readers-Writers Problem. Discuss the first Readers-Writers solution and explain how starvation may occur.

7. Explain the Dining Philosophers Problem with a neat diagram. Discuss why deadlock and starvation may occur.

8. Five autonomous robots share five rechargeable battery docks arranged in a circular manner. Model this scenario using the Dining Philosophers Problem and design a synchronization strategy that prevents deadlock.

Lecture 11 – Principles of Deadlock

1. Define deadlock. State the four necessary conditions required for a deadlock to occur.

2. Differentiate between deadlock prevention and deadlock avoidance.

3. What is a safe state? How does it differ from an unsafe state?

4. A system currently satisfies all four Coffman conditions. Does this necessarily imply that a deadlock has occurred? Justify.

5. Explain the four necessary conditions for deadlock with suitable examples.

6. Illustrate a Resource Allocation Graph involving four processes and four resources. Identify whether the system is in a deadlock state.

7. Explain the working principle of Banker's Algorithm. Describe how the Available, Allocation, Maximum, and Need matrices are used.

8. Using a given system state with Allocation, Maximum, and Available matrices, determine whether the system is in a safe state using Banker's Algorithm.

9. Explain the methods for handling deadlocks: prevention, avoidance, detection, and recovery.

10. Describe how each of the four necessary conditions for deadlock can be prevented.

11. Explain the Resource Allocation Graph algorithm for deadlock avoidance. How does a claim edge differ from a request edge?

12. Differentiate between the wait-for graph and the resource allocation graph. When is the wait-for graph sufficient for deadlock detection?

13. Compare Deadlock Prevention, Deadlock Avoidance, Deadlock Detection, and Deadlock Recovery with respect to implementation complexity, runtime overhead, and resource utilization.

14. A cloud server hosts multiple virtual machines competing for CPU cores, memory pages, and storage devices. Analyze whether a deadlock can exist and propose suitable strategies.

15. A distributed banking system processes thousands of concurrent financial transactions. Design a deadlock management strategy combining prevention, avoidance, and recovery techniques.

16. What is the difference between a safe state and an unsafe state? Does an unsafe state always lead to deadlock?

Lecture 13 – Signals and Signal Handling

1. Define a signal in Linux. Why are signals considered an asynchronous communication mechanism?

2. Differentiate between SIGINT and SIGTERM.

3. State the purpose of the kill() and signal() system calls.

4. A user accidentally presses Ctrl+C while a program is executing. Explain how the operating system handles this event.

5. Explain the life cycle of a signal from its generation to its handling by a process.

6. Write a C program using the signal() function to catch the SIGINT signal and display a custom message.

7. Differentiate between SIGINT, SIGTERM, SIGKILL, and SIGSTOP. Explain their purpose and typical use cases.

8. Explain the advantages and limitations of signal-based communication compared with pipes and shared memory.

9. Develop a Linux program where a parent process sends SIGUSR1, SIGUSR2, and SIGTERM to a child process using kill(). The child should handle each signal differently.

10. A multithreaded Linux server must terminate gracefully when it receives SIGINT or SIGTERM. Design a signal-handling strategy that ensures proper resource cleanup.

11. Compare signal-based communication, pipes, shared memory, and message passing with respect to communication model, synchronization, performance, reliability, and implementation complexity.

12. Which signals cannot be caught, blocked, or ignored? Why does the OS enforce this restriction?

13. Explain the difference between default signal handling and custom signal handling using the signal() function.

14. What happens when a process receives a signal while it is blocked in a system call? Discuss the concept of signal interruption.